IEEE SOLID-STATE CIRCUITS SOCIETY
IC Innovation
Alexandria Student Chapter | Alexandria Student Branch

# Session 1

*Java fundamentals*

# Agenda



1. Java Overview
2. Program Structure
3. Variables & Data Types
4. Type Casting
5. Operators
6. Input (Scanner)
7. Conditionals (if / switch)
8. Loops (for / while / do-while)
9. Arrays
10. Basic Methods
11. Common Errors
12. Practice Exercises

SECTION 1

# Java Overview

*What is Java, Why Java, JVM & Compilation*

# What is Java?

- High-level, general-purpose, object-oriented programming language
- Created by James Gosling at Sun Microsystems (1995)
- Runs on billions of devices: phones, servers, embedded systems
- Strongly typed: every variable must have a declared type
- Compiled AND interpreted: source code → bytecode → JVM executes

Created: 1995

#1 Enterprise Language

Android Platform

Runs on 3+ Billion devices

Memory Safe

# Why Learn Java?



### Platform Independent

Write Once, Run Anywhere (WORA)

### Huge Community

Massive ecosystem, libraries & tools

### In-Demand

Top language in enterprise & Android dev

### Readable Syntax

Clear structure helps learn programming

### OOP Ready

Best intro language before advanced OOP

# JVM – Java Virtual Machine

*The JVM is a virtual computer that runs Java programs — it's what makes Java platform-independent!*

- JVM reads bytecode, not your OS's native code
- Same .class file runs on Windows, Mac, Linux — as long as JVM is installed
- JVM also handles memory management (Garbage Collection)

# Compilation Process



**Source Code (.java)** → **Java Compiler (javac)** → **Bytecode (.class)** → **JVM** → **Output**

```
// 1. You write: Hello.java
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello!");
  }
}
```

```
# 2. Compile:
javac Hello.java  →  Hello.class

# 3. Run:
java Hello  →  Hello!
```

SECTION 2

# Program Structure

*File structure, main method, output, comments*

# Basic File Structure



```java
// File name: Hello.java
public class Hello {

    // main method - entry point
    public static void main(String[] args) {

        // Print output
        System.out.println("Hello, World!");

    }

}
```

- File name MUST match class name
- class keyword defines a class
- main() is where Java starts running
- Every statement ends with ;
- Blocks use { curly braces }

# main Method & System.out.println



## The main method signature (must be EXACT):

```
public static void main(String[] args) { ... }
```

## Output Methods:

```
System.out.println("Hello!");     // prints + new line
System.out.print("Hello ");       // prints WITHOUT new line
System.out.println(42);           // prints a number
System.out.println(3 + 4);        // prints result of expression: 7
System.out.println("2 + 2 = " + (2+2));  // prints: 2 + 2 = 4
```

# Comments in Java



```
// This is a SINGLE-LINE comment – ignored by the compiler

/*
   This is a MULTI-LINE comment.
   Use it for longer explanations.
*/

/**
 * This is a JAVADOC comment.
 * Used to generate documentation.
 * @param name  The person's name
 */
public static void greet(String name) { ... }
```

- Comments are NOT executed — only for humans reading the code
- Use comments to explain WHY, not WHAT (code shows what)
- Good habit: comment complex logic, not obvious lines

SECTION 3

# Variables & Data Types

*Primitives, declaration, initialization, naming rules*

# Primitive Data Types



| Type | Description | Range | Example |
|---|---|---|---|
| **int** | 32-bit integer | -2B to 2B | `int age = 20;` |
| **double** | 64-bit decimal | ~15 digits | `double pi = 3.14;` |
| **float** | 32-bit decimal | ~7 digits | `float x = 3.14f;` |
| **long** | 64-bit integer | Very large | `long big = 9L;` |
| **char** | Single character | Unicode | `char c = 'A';` |
| **boolean** | true / false | 1 bit | `boolean ok = true;` |
| **byte** | 8-bit integer | -128 to 127 | `byte b = 100;` |
| **short** | 16-bit integer | -32K to 32K | `short s = 200;` |

# Declaration & Initialization



```
// Declaration only (no value yet)
int score;

// Declaration + Initialization
int score = 100;
double temperature = 36.6;
char grade = 'A';
boolean isStudent = true;
String name = "Alice";     // String is NOT primitive (it's a class)

// Multiple variables of same type
int x = 5, y = 10, z = 15;
```

- Always initialize before using a variable (or you'll get a compile error!)
- String starts with capital S — it's a special class, not a primitive
- String literals use double quotes " ", char uses single quotes ' '

# Variable Naming Rules



## ✅ Good Naming

- camelCase for variables: studentAge, totalScore
- Start with letter, _ or $
- Descriptive names: totalMarks not tm
- Constants: ALL_CAPS → final int MAX_SIZE = 100;

## ❌ Naming Mistakes

- Start with a digit: 1name ❌
- Spaces in names: my age ❌
- Java keywords: int, class, public ❌
- Special chars (except _ and $): my@name ❌

```
int myAge = 20;          // ✅ camelCase
final double PI = 3.14;  // ✅ constant
int 2fast = 10;          // ❌ starts with digit
```

SECTION 4

# Type Casting

*Implicit & Explicit conversion between types*

# Type Casting – Implicit & Explicit



**Implicit (Widening) — automatic, no data loss:**

```
int num = 42;
double result = num;   // int → double automatically (safe!)
System.out.println(result);  // 42.0
```

**Explicit (Narrowing) — manual, may lose data:**

```
double pi = 3.99;
int truncated = (int) pi;   // explicit cast — drops decimal
System.out.println(truncated);  // 3  (NOT 4 — it truncates, not rounds!)

char ch = 'A';
int code = ch;              // char → int  →  65 (ASCII value)
```

*Widening path: byte → short → int → long → float → double*

**Exercise**

What is the output? double d = 9.7; int i = (int) d; System.out.println(i);

SECTION 5

# Operators

*Arithmetic, Assignment, Comparison, Logical, Inc/Dec*

# Arithmetic Operators



```java
int a = 10, b = 3;

System.out.println(a + b);  // 13
System.out.println(a - b);  // 7
System.out.println(a * b);  // 30
System.out.println(a / b);  // 3 (integer division!)
System.out.println(a % b);  // 1 (remainder/modulo)

double x = 10.0 / 3.0;
System.out.println(x);      // 3.3333...
```

| | | |
|---|---|---|
| + | Addition | `5 + 3 = 8` |
| - | Subtraction | `5 - 3 = 2` |
| * | Multiplication | `5 * 3 = 15` |
| / | Division | `10 / 3 = 3` |
| % | Modulo (remainder) | `10 % 3 = 1` |

⚠ ***Integer division: 10/3 = 3 (not 3.33). Use double for decimals!***

# Assignment & Comparison Operators



## Assignment Operators:

```
int x = 10;   // assign 10 to x
x += 5;       // x = x + 5  →  15
x -= 3;       // x = x - 3  →  12
x *= 2;       // x = x * 2  →  24
x /= 4;       // x = x / 4  →  6
x %= 4;       // x = x % 4  →  2
```

## Comparison Operators (return boolean):

```
int a = 5, b = 8;
a == b  // false (equal?)
a != b  // true  (not equal?)
a > b   // false (greater than?)
a < b   // true  (less than?)
a >= b  // false
a <= b  // true
```

***⚠ Don't confuse = (assign) with == (compare equal)!***

```
// This is ASSIGNMENT:     x = 5   (sets x to 5)
// This is COMPARISON:     x == 5  (asks: is x equal to 5? → true/false)
```

# Logical & Increment/Decrement Operators



## Logical Operators:

```
boolean a = true, b = false;
System.out.println(a && b);  // AND  →  false (both must be true)
System.out.println(a || b);  // OR   →  true  (at least one true)
System.out.println(!a);      // NOT  →  false (flips true/false)
```

## Increment / Decrement:

```
int x = 5;
x++;        // x becomes 6  (post-increment: use then add)
x--;        // x becomes 5  (post-decrement: use then subtract)
++x;        // x becomes 6  (pre-increment: add then use)

// Difference:
int y = 5;
System.out.println(y++);  // prints 5, THEN increments → y is now 6
System.out.println(++y);  // increments FIRST → prints 7
```

SECTION 6

# Input with Scanner

*import, reading different types, common mistakes*

# Reading Input – Scanner Class

```
import java.util.Scanner;   // ← must be at TOP of file

public class InputDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);  // create scanner

        System.out.print("Enter your name: ");
        String name = sc.nextLine();     // reads a whole line

        System.out.print("Enter your age: ");
        int age = sc.nextInt();          // reads an integer

        System.out.print("Enter GPA: ");
        double gpa = sc.nextDouble();    // reads a decimal

        System.out.println("Hello " + name + ", age " + age + ", GPA " + gpa);
        sc.close();   // good practice to close scanner
    }
}
```

*Key Scanner Methods: nextLine() · nextInt() · nextDouble() · nextFloat() · next() · nextBoolean()*

# Scanner – Common Mistakes



## 🐛 Bug: Mixing nextInt() and nextLine()

```
int age = sc.nextInt();   // reads number, leaves '\n' in buffer
String name = sc.nextLine(); // reads the leftover '\n' → name is EMPTY! ❌

// FIX: add an extra nextLine() to consume the newline:
int age = sc.nextInt();
sc.nextLine();            // flush the buffer ← add this line
String name = sc.nextLine(); // now reads correctly ✅
```

**Other Common Mistakes:**

- Forgetting import java.util.Scanner; at the top
- Calling nextInt() but entering text → InputMismatchException
- Not creating Scanner: Scanner sc = new Scanner(System.in);
- Using wrong method: nextLine() when you need nextInt()

SECTION 7

# Conditionals

*if / if-else / else-if / nested if / switch*

# if / if-else Statement



```java
// Basic if
int score = 75;
if (score >= 60) {
    System.out.println("Pass!");
}

// if-else
if (score >= 60) {
    System.out.println("Pass!");
} else {
    System.out.println("Fail.");
}
```

```java
// else-if chain
int marks = 82;

if (marks >= 90) {
    System.out.println("A");
} else if (marks >= 80) {
    System.out.println("B");  // ← this runs
} else if (marks >= 70) {
    System.out.println("C");
} else {
    System.out.println("F");
}
```

*Java evaluates conditions top-to-bottom and stops at the first true branch.*

# Nested if & Switch Statement



```java
// Nested if
int age = 20;
boolean hasID = true;

if (age >= 18) {
    if (hasID) {
        System.out.println("Entry allowed");
    } else {
        System.out.println("Need ID");
    }
} else {
    System.out.println("Too young");
}
```

```java
// switch statement
int day = 3;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    case 3: System.out.println("Wed"); break;
    default: System.out.println("Other");
}
```

⚠ ***Always add break; in switch cases to prevent fall-through to next case!***

**Exercise**

Write a program that reads a number and prints:
"Positive" if > 0, "Negative" if < 0, "Zero" if == 0.
Bonus: Using switch, print the name for numbers 1-5 (e.g., 1 → "One").

SECTION 8

# Loops

*for / while / do-while / break / continue / nested*

# for Loop



**Syntax: for (initialization ; condition ; update) { body }**

```
// Print 1 to 5
for (int i = 1; i <= 5; i++) {
    System.out.println("i = " + i);
}

// Print even numbers 2 to 10
for (int i = 2; i <= 10; i += 2) {
    System.out.print(i + " ");   // output: 2 4 6 8 10
}

// Count DOWN from 5 to 1
for (int i = 5; i >= 1; i--) {
    System.out.print(i + " ");   // output: 5 4 3 2 1
}
```

- Init: runs once at start
- Condition: checked before each iteration
- Update: runs after each iteration body

# while & do-while Loops



```java
// while: checks BEFORE running
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
// Use when: number of iterations unknown
```

```java
// do-while: runs AT LEAST ONCE
int num = 1;
do {
    System.out.println(num);
    num++;
} while (num <= 5);
// Runs body THEN checks condition
```

**When to use which loop?**

- for → when you know the exact number of iterations
- while → when you don't know how many times (e.g., user keeps entering input)
- do-while → when you need the body to execute at least once (e.g., menu)

# break, continue & Nested Loops



```java
// break: exits the loop
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;
    System.out.print(i + " ");
}
// Output: 1 2 3 4

// continue: skips this iteration
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.print(i + " ");
}
// Output: 1 2 4 5
```

```java
// Nested loops: multiplication table
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i*j + "\t");
    }
    System.out.println();
}
/* Output:
1  2  3
2  4  6
3  6  9  */
```

**Exercise**

1) Write a for loop that prints all multiples of 3 from 3 to 30.
2) Write a loop that keeps asking the user for a number until they enter 0.
3) Print the following pattern using nested loops:

```
*
**
***
```

SECTION 9

# Arrays

*Declaration, accessing elements, looping, common errors*

# Arrays – Declaration & Access



```
// Declare and initialize
int[] grades = {85, 90, 78, 92, 88};

// Declare with size (all zeros by default)
int[] scores = new int[5];

// Access by index (starts at 0!)
System.out.println(grades[0]);   // 85  (first element)
System.out.println(grades[4]);   // 88  (last element)
System.out.println(grades.length); // 5 (total count)

// Modify an element
grades[2] = 95;   // now: {85, 90, 95, 92, 88}
```

| Index: | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| Value: | 85 | 90 | 78 | 92 | 88 |

⚠ ***Index starts at 0, last index = length - 1***

# Looping Through Arrays & Common Errors



```java
// Standard for loop
int[] nums = {10, 20, 30, 40, 50};

for (int i = 0; i < nums.length; i++) {
    System.out.println(nums[i]);
}

// Enhanced for-each loop
for (int num : nums) {
    System.out.println(num);
}
```

```java
// Find sum of array
int[] nums = {3, 7, 2, 9, 5};
int sum = 0;

for (int n : nums) {
    sum += n;
}
System.out.println("Sum: " + sum);  // 26

// Find max value
int max = nums[0];
for (int n : nums) {
    if (n > max) max = n;
}
System.out.println("Max: " + max); // 9
```

## Common Array Errors:

- ArrayIndexOutOfBoundsException: accessing index >= length (e.g., arr[5] on a 5-element array)
- Off-by-one: loop uses i <= arr.length instead of i < arr.length

SECTION 10

# Basic Methods

*Syntax, parameters, return types, void methods*

# Method Syntax & Structure



## Anatomy of a method:

```
//  access   static   returnType  methodName  ( parameters )
    public   static   int         add         ( int a, int b )  {
        return a + b;   // must return int
    }
```

```
// void method (no return value)
public static void greet(String name) {
    System.out.println("Hello, " + name + "!");
}

// Calling it:
greet("Alice");   // prints: Hello, Alice!
```

```
// method with return value
public static int square(int n) {
    return n * n;
}

// Calling it:
int result = square(5);
System.out.println(result);  // 25
```

*Methods let you reuse code: write once, call many times!*

# Method Examples



```java
public class MethodDemo {

    // Returns the larger of two numbers
    public static int max(int a, int b) {
        if (a > b) return a;
        return b;
    }

    // Checks if a number is even
    public static boolean isEven(int n) {
        return n % 2 == 0;
    }

    public static void main(String[] args) {
        System.out.println(max(10, 20));      // 20
        System.out.println(isEven(4));        // true
        System.out.println(isEven(7));        // false
    }
}
```

**Exercise**

Write a method sumArray(int[] arr) that takes an integer array and returns the sum of all elements.

SECTION 11

# Common Errors

*Syntax, Logical, Runtime — understand before you debug!*

# Types of Errors in Java



## Syntax Error

*Caught by the compiler — program won't even run*

Missing semicolon: `System.out.println("Hi")` · Wrong case: `system.out.println` (should be System) · Missing closing brace `}`

## Runtime Error

*Happens while running — crashes the program*

Division by zero: `int x = 5/0` → ArithmeticException · Null pointer: `String s = null; s.length()` → NullPointerException · Array out of bounds: `arr[10]` on 5-element array

## Logical Error

*Program runs, but gives WRONG output — hardest to find!*

Using `=` instead of `==` in a condition · Off-by-one in loops: `<` vs `<=` · Wrong formula: using `+` instead of `*`

SECTION 12

# Practice Exercises

*Put it all together — code time!*

# Quick Reference – Syntax Cheat Sheet



```java
// Variables
int x = 5;  double d = 3.14;
char c = 'A';  boolean b = true;
String s = "Hello";

// Input
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();

// Output
System.out.println("val: " + n);

// Conditions
if (x > 0) { ... }
else if (x == 0) { ... }
else { ... }

// Switch
switch (x) {
  case 1: ... break;
  default: ...
}
```

```java
// for loop
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

// while loop
while (x > 0) { x--; }

// do-while
do { ... } while (condition);

// Arrays
int[] arr = {1, 2, 3};
arr[0] = 10;

// for-each
for (int val : arr) { ... }

// Method
public static int add(int a, int b){
    return a + b;
}
```

# Session Summary



### ☕ Java Overview

JVM, bytecode, WORA, compilation

### 🏗️ Program Structure

class, main(), print, comments

### 📦 Variables & Types

8 primitives + String, casting

### ➕ Operators

Arithmetic, assignment, compare, logical

### ⌨️ Input

Scanner, import, common pitfalls

### 🔀 Conditionals

if/else-if/switch/nested

### 🔁 Loops

for/while/do-while, break/continue

### 📋 Arrays

Declare, access, loop, errors

### 🛠️ Methods

void, return type, parameters

### 🐛 Errors

Syntax vs Runtime vs Logical

# References



**JVM, JRE, and JDK - Fully Explained in 5 Minutes (Coding with John)**

https://www.youtube.com/watch?v=KctLuhwFEQ8

**Java Full Course for Beginners (Programming with Mosh)**

https://www.youtube.com/watch?v=eIrMbAQSU34&t=378s

**Java Programming For Beginners - Course 1- بالعربى (Mohamed El Desouki)**

https://www.youtube.com/playlist?list=PL1DUmTEdeA6K7rdxKiWJq6JIxTvHalY8f

# Thank You!

*Keep coding — practice makes perfect* ☕

IEEE-SSCS | Java Basic Syntax Session